AgentMPI: A Message Passing Interface for Portable Multi-Agent Harnesses

Anonymous research artifact

Research draft — 30 August 2026

Abstract

Multi-agent language-model systems are commonly assembled as bespoke orchestration graphs. Their prompts, queues, failure behavior, context management, and communication topology are entangled with a particular framework. This is analogous to parallel computing before the Message Passing Interface (MPI), when portable applications were impeded by incompatible vendor libraries. We present AgentMPI, a transport-neutral interface for writing multi-agent harnesses. AgentMPI adapts MPI's communication contexts, ranks, tagged matching, requests, collectives, and implementation-independent semantics, while rejecting an unsafe literal analogy between deterministic HPC processes and expensive, nondeterministic, fallible agents. It adds executor incarnations, membership generations, revocation and shrink, context credits, content-addressed artifacts, semantic acknowledgments, capability boundaries, and fenced leases.

We define point-to-point and collective semantics, connect each operation to an executable implementation, and provide a durable SQLite reference runtime for independent processes. Its roughly 1.5 k-line core persists protocol state in short transactions and supports four delivery modes, wildcard matching, eight collectives, heartbeat failure detection, communicator repair, bounded mailboxes, artifact spill, and append-only traces. In a 20-trial local microbenchmark, fail-and-shrink of an eight-rank communicator takes a median 1.36 ms; externalizing a 16,253-token payload leaves a 36-token mailbox reference with byte-identical resolution. SQLite collective latency rises from 5.58 ms at two ranks to 476 ms at 32 ranks, confirming that this binding is an inspectable reference rather than a cluster transport. A separate process test completed correct allreduce and fan-in with 100 independent ranks; in a 16-process fail-stop test, rank 7 exited with status 99 and rank 0 explicitly marked it failed; all 15 survivors observed revocation, agreed on membership, and completed post-repair allreduce.

We additionally execute reproducible book-translation and collaborative software-development workloads with 25 separately launched Cursor subagent executors communicating through AgentMPI; a retained manifest records every executor identifier and excludes coordinators. These runs are pilot evidence for implementability, not a claim that agent count alone improves quality. The result is a concrete protocol, artifact, and evaluation agenda for separating multi-agent program logic from rapidly changing agent infrastructure.

1 Introduction

An MPI program says that rank 0 broadcasts parameters, ranks compute, and a reduction combines results. It does not say whether bytes traverse shared memory, TCP, InfiniBand, UCX, or a vendor interconnect. That separation let a portable programming model survive several decades of machine evolution. Current multi-agent harnesses lack an equivalent waist. A "team" is often a framework-specific set of Python objects; a "message" may be a list append, a prompt fragment, or an HTTP task; and "memory" may mean both durable state and text copied into every model context.

The analogy is useful but dangerous. An MPI process is comparatively cheap, deterministic between communications, and placed in a trusted allocation. An LLM agent can take minutes, return plausible nonsense, exceed a context window, pause for a tool or person, and execute outside the operator's trust domain. A barrier can prove that every executor called the barrier; it cannot prove that every model understood a policy. A broadcast can deliver identical bytes while attention and interpretation differ. Natural-language synthesis is usually not an associative reduction.

We therefore ask a narrower systems question:

> Can a protocol preserve MPI's explicit and portable coordination semantics while making agent-specific uncertainty, context, lifecycle, and failure visible rather than pretending them away?

This paper answers with AgentMPI, an interface rather than a prescribed multi-agent system. Its intended user is a harness author. Implementers can map it to local tasks, a durable broker, actor mailboxes, A2A, or provider APIs. Programs depend on observable semantics; implementations choose transports and algorithms.

The contributions are:

1. a systematic translation of MPI objects and operations into agent-harness semantics, including explicit statements of where the analogy fails;
2. point-to-point matching, completion, progress, collective, lifecycle, failure, context, synchronization, security, and tracing contracts;
3. a durable, brokerless Python/SQLite reference implementation and canonical JSON envelope schema;
4. conformance tests, fault hooks, and reproducible microbenchmarks grounded in machine-readable traces;
5. true multi-agent translation and software-development case studies, plus an evaluation design that separates protocol mechanics from model quality.

This is a first research artifact, not a completed standard. In particular, one SQLite implementation cannot demonstrate interoperability, and pilot agent runs cannot establish a quality scaling law. We state those boundaries because systems interfaces fail when aspirational semantics are reported as measured behavior.

2 What MPI Actually Contributed

2.1 Before and through MPI-1

MPI emerged from a fragmented environment of vendor and research message-passing systems. The 1992–1993 forum brought vendors, library implementers, and application authors together; over forty organizations participated, and MPI-1.0 was finalized in May 1994 [32, 33]. The achievement was not invention of send, receive, or barrier. It was agreement on a sufficiently precise, language-independent library contract that admitted efficient implementation on heterogeneous distributed-memory machines.

Four philosophical choices matter to AgentMPI.

Library, not language. MPI did not require a new parallel language. Existing C and Fortran programs could adopt a stable interface incrementally. An agent protocol should similarly avoid requiring one workflow DSL or one model framework.

Communication context plus group. A communicator combines ordered membership with a private matching namespace. This is more than a list of workers: independently developed libraries can use the same tags without stealing each other's messages. Rank is local to that context, not a global identity.

Semantic room for optimization. MPI defines when operations match and complete but leaves eager versus rendezvous protocols, buffering, progress, topology selection, and transport to implementations. MPICH demonstrated that an abstract device layer could be both portable and performant [19]; Open MPI later emphasized a runtime-selected modular component architecture [17]. AgentMPI similarly separates protocol, executor adapter, transport, artifact store, and control plane.

Collectives as first-class intent. A broadcast expressed as $P-1$ sends hides optimization opportunities and coordination intent. A collective call exposes both. MPI implementations can select binomial trees, rings, recursive doubling, or topology-aware algorithms without changing application code [39].

2.2 Evolution: MPI-2 through MPI-4.1

MPI-2 added dynamic processes, one-sided communication, parallel I/O, and other facilities; MPI-3 expanded nonblocking collectives, neighborhood collectives, and shared-memory windows; MPI-4 added Sessions, partitioned communication, and persistent collectives. Version 4.1, approved 2 November 2023, is primarily a clarification release [33]. Two lessons follow.

First, a small stable core can precede richer resource and topology facilities. AgentMPI defines conformance levels instead of claiming every mature MPI feature in version 0.1. Second, global initialization became a composability problem: Sessions let components obtain process sets and construct communication contexts without assuming one global `WORLD`. Agent harnesses need this from the beginning because nested teams and tools often have independent lifecycles.

MPI-5.0, approved 5 June 2025, added a standard ABI and a small number of targeted changes rather than a new programming model [34]. In particular, the released standard does not include revoke, shrink, agree, or the ULFM failure error classes; those remain research/"MPI next" work. Keeping that distinction matters for AgentMPI: ULFM is a design source, not a feature that can be attributed to the current MPI standard.

2.3 MPI's fault boundary

Traditional MPI was optimized for tightly managed, mostly fail-stop jobs; a single process failure often made continued communication impractical. FT-MPI, MPICH-V, and User-Level Failure Mitigation (ULFM) investigated recovery [6, 5]. ULFM's key abstraction is modest: repair communication capability through failure acknowledgment, revoke, agreement, and communicator shrink; application state recovery remains the application's responsibility [4]. This distinction is central for agents. A protocol can reassign a task and restore a mailbox; it cannot reconstruct unrecorded reasoning or undo an unfenced side effect.

MPI runtime work also separated launch/resource management from communication. PMIx offers a programming-model-neutral process-management interface [9]. AgentMPI follows that split: an executor

adapter launches or invokes agents, while the communication layer owns matching and collectives.

3 Why Existing Agent Interfaces Are Insufficient

3.1 Frameworks are programs, not a portable waist

AutoGen models applications as conversations among configurable agents [41]; CAMEL uses role-playing communication [29]; MetaGPT and ChatDev organize software-development roles and artifacts [25, 36]. These systems provide useful orchestration policies. They do not define a common matching, completion, collective, failure, context-credit, and repair contract that an independently written harness can carry between runtimes.

This distinction mirrors the difference between a parallel application framework and MPI. AgentMPI should be able to express a debate, software company, tree of reviewers, or simple ensemble; it should not standardize any one of them.

3.2 Interoperability protocols address adjacent layers

FIPA ACL standardizes message fields and communicative acts [16]. MCP standardizes host/client/server exchange for tools, resources, and prompts [35]. A2A 1.0 targets interoperability among opaque agentic services and defines discovery, tasks, messages, artifacts, and multiple protocol bindings [1]. These are complementary: an AgentMPI transport may carry envelopes over A2A, and an AgentMPI executor may use MCP tools. Neither currently supplies MPI-like communicator isolation, ranked collectives, stream ordering, collective mismatch, context credits, or ULFM-like repair.

3.3 More communication is not automatically better

Sampling and voting can improve benchmark accuracy as ensemble size grows [30], and layered mixture-of-agents systems report quality gains [40]. But carefully controlled studies find that debate does not reliably beat simpler self-consistency or voting [38, 12]. Similar agents can converge to a shared misconception [14]; long interactions can drift away from the problem [3]. The systems implication is not to encode "debate" as a blessed primitive. It is to provide measurable communication topologies and make simpler baselines easy.

4 Requirements and Failure Model

AgentMPI targets heterogeneous, high-latency executors connected by an asynchronous transport. The core assumes crash-stop executor failures and non-Byzantine control metadata. A lease detector may eventually suspect a silent executor, but in an asynchronous system silence cannot distinguish failure, partition, and delay [11]. Byzantine model outputs—including prompt injection and confidently wrong text—are application data requiring validation and provenance.

The design requirements are:

Isolation. Concurrent libraries and teams cannot consume each other's messages.

Explicit nondeterminism. Wildcard matches, retries, and model sampling are traced; deterministic replay does not falsely promise identical generation.

Resource bounds. Mailbox bytes, inline tokens, context materialization, and provider capacity have admission rules.

Failure visibility. Strict collectives never silently drop a missing rank; membership changes create a new generation.

Transport independence. A binding publishes capability, progress, delivery, and durability levels instead of leaking its queue implementation.

Semantic humility. Delivery, match, and model acceptance are distinct.

Security. A communicator is a capability boundary, not only an integer namespace.

5 Protocol Objects

5.1 Session, executor, and incarnation

A session isolates agents, communicators, messages, artifacts, locks, and a trace. An *agent* is a logical identity; an *executor* is the process, remote service, human, or model invocation acting for it. Restarting an executor increments an incarnation. Delayed messages and commits from an older incarnation can then be rejected.

The lifecycle is

$$\textsc{Joining} \to \textsc{Active} \to \textsc{Draining} \to \textsc{Finalized},$$

with $\textsc{Active} \to \textsc{Suspect} \to \textsc{Failed}$ and a new incarnation returning through Joining. Transitions are monotonic inside an incarnation.

5.2 Communicator and membership generation

A communicator is

$$C = (\mathit{session}, \mathit{context}, \mathit{generation}, \langle \mathit{members} \rangle, \mathit{attributes}).$$

Rank is the index/name within the ordered membership. The context isolates matching; the generation identifies

an immutable membership epoch. Split, grow, and shrink return a new object and an explicit old-to-new map. No implementation may silently mutate membership behind a rank's handle.

5.3 Envelope and status

The production JSON schema accompanies the artifact. Identity and matching fields include protocol version, message ID, session, communicator, generation, source rank and incarnation, destination, tag, and per-stream sequence. The body is either inline typed JSON or a content-addressed artifact reference. Resource and provenance fields include bytes, estimated tokens, schema, deadline, trace context, capability proof, checksum, and creation time.

The receiver validates identity, generation, destination, authorization, and integrity before exposure. Status returns the actual source/tag selected by a wildcard, sequence, message ID, byte/token charge, and artifact reference.

6 Point-to-Point Semantics

6.1 Matching and non-overtaking

`Send(v,d,t,C)` matches `Recv(s,t,C)` when communicator, generation, destination, source constraint, and tag constraint agree. `ANY_SOURCE` and `ANY_TAG` are explicit wildcards. The earliest eligible admitted envelope wins.

Non-overtaking is deliberately narrow. For fixed (C, g, s, d, t), if send m_1 precedes m_2, a receive that can match both cannot return m_2 first. There is no total order across sources or tags. Lamport clocks can represent causality [27], but are metadata, not hidden matching rules. Wildcard match decisions are logged because they make replay schedule-dependent.

6.2 Four completion points

Agent communication needs more distinctions than a Boolean "sent":

1. *local*: caller state can be reused;
2. *admitted*: the destination transport accepted responsibility;
3. *matched*: a posted destination receive selected the message;
4. *accepted*: destination application returned a typed semantic ACK.

Standard/buffered send completes at admission; synchronous send waits for match. Ready send is legal only if a compatible receive is already posted. None implies correct execution or durable external side effects. Exactly-once model execution is not generally available: a provider may finish before its ACK is lost. Durable bindings use stable message/task IDs and idempotent, fenced commits.

6.3 Requests, progress, and cancellation

Nonblocking operations return requests observed by `Test`, `Wait`, `WaitAny`, `WaitSome`, and `WaitAll`. A binding declares weak progress (calls/polling required) or strong progress (background progress). Cancellation is an attempt: cancelling a local request cannot retract an admitted task or prove cancellation of a remote model.

Our v0.1 reference binding implements blocking operations, probe, and posted receives. Nonblocking requests are specified but intentionally not claimed as implemented; this prevents a thread wrapper from being presented as a portable asynchronous progress guarantee.

7 Collectives and Their Agent Analogues

Collectives are ordered within a communicator generation. Every live rank must enter the same operation signature at the same ordinal. A mismatch is an error, not a different legal collective. Table 1 gives semantics and algorithms; optimized algorithms preserve results and completion.

7.1 Barrier

At dissemination round k, rank r sends to $(r+2^k) \bmod P$ and receives from $(r - 2^k) \bmod P$. It takes $\lceil \log_2 P \rceil$ rounds without a central hot spot. Agent phases may last minutes, so barriers need deadlines and straggler diagnostics. A strict barrier with a failed member fails; a quorum phase is a different operation that returns its participant set.

7.2 Broadcast and context

A policy or glossary should be stored once, addressed by digest, then broadcast as a reference. Each adapter selectively resolves it when constructing an invocation. An application that needs evidence of parsing follows broadcast with structured validation and `Allreduce(AND)`. Even then the result means all validators returned true, not that the policy is true.

7.3 Reduction versus synthesis

A reduction declares input/output schema, identity, associativity, commutativity, conflict policy, determinism, and implementation version. Numeric sum and Boolean conjunction qualify. LLM "merge these drafts" normally does not: output depends on rank order, prompt, model, and sampling. AgentMPI represents it honestly as `Gather` followed by a designated synthesis task. This preserves optimization safety and makes model cost visible.

Operation	Agent meaning	What it does *not* mean	Candidate algorithm
Barrier	No rank exits phase k before every live member enters it.	Prompts understood; effects flushed; state agreed.	Dissemination: $\lceil \log_2 P \rceil$ rounds [21].
Bcast	Root delivers one immutable policy, schema, glossary, or artifact digest to all ranks.	Identical attention or interpretation.	Binomial tree for small values; scatter+allgather for large.
Scatter	Root assigns one rank-indexed shard.	Shards are independent or balanced.	Binomial/scattered tree.
Gather	Root obtains rank-ordered outputs.	Outputs are mutually consistent.	Linear for small P; binomial tree.
Allgather	Every rank obtains all rank-ordered summaries/proposals.	All full artifacts should enter every prompt.	Ring for large values; recursive doubling for small.
Reduce	Deterministic associative operator combines typed values.	Arbitrary LLM synthesis is associative.	Trees, recursive doubling, Rabenseifner [37].
Allreduce	Reduction result reaches all ranks.	Majority output is true or consensus-safe.	Reduce-scatter+allgather or recursive doubling.
Alltoall	Personalized exchange, e.g., assigned peer reviews.	Every agent should converse with every peer.	Pairwise/radix/Bruck [7].
Scan	Rank i receives prefix reduction through i.	Later stages may mutate earlier committed state.	Recursive doubling.
Neighborhood	Declared graph neighbors exchange values.	The graph captures all semantic dependencies.	Topology-specific.

Table 1: MPI collectives translated into precise agent-harness semantics.

7.4 Agent-aware cost model

The Hockney-style network model motivates but does not describe agents. We use

$$T_{\text{op}} = \alpha_h + \alpha_i + \beta_b n_{\text{bytes}} + \beta_t n_{\text{tokens}} + \gamma_m(x) + T_q + T_r,$$

where α_h is harness startup, α_i invocation startup, β_b transport cost, β_t context ingestion, γ_m task/model generation, T_q queueing/backpressure, and T_r expected recovery. Token volume can change result quality as well as latency, so an implementation reports transport and model intervals separately. Hierarchical collectives can summarize within cheap/local groups before crossing model or organization boundaries. Section 8 develops the special case of this model that governs reductions, where the operator's declared algebra fixes the depth term and the rank context bound turns a cost comparison into a feasibility test.

8 The Algebra of Agent Reductions

Naming an operation `allreduce` is worth something only if the runtime is free to choose how to perform it. MPI bought that freedom with one sentence in the standard: the operator passed to a reduction must be associative, and the implementation may therefore evaluate it in any order consistent with associativity, and with commutativity when the user declares it through `MPI_Op_create`'s `commute` flag [33]. Thirty years of collective optimisation — recursive doubling, Rabenseifner's reduce-scatter/allgather decomposition, ring algorithms, hardware offload — all sit inside the licence that sentence grants [39, 37, 10].

The freedom is not free of consequence even in MPI. Because reordering is permitted, a floating-point sum is not bitwise reproducible across process counts, and the standard declines to promise that it will be. Practitioners accept this because the error is bounded and well understood.

For agents the same trade appears with the bound removed. The reductions a harness actually wants — merge these partial glossaries, reconcile these two API designs, synthesise these eight reviews — are evaluated by a language model. Such an operator is at best approximately associative: merging A into B and then C need not produce what merging B into C and then A produces, and the difference is not a rounding error but a different document.

AgentMPI therefore makes the algebra an explicit, machine-read part of the operator declaration, and lets it *constrain the algorithm*:

declared	admissible schedules	depth
assoc. and comm.	any tree, any order, block-decomposed	$O(\log p$
assoc. only	any tree, rank order	$O(\log p$
neither	canonical linear order	$O(p)$

The third row is the interesting one, and it is what MPI never had to write down because MPI simply refused non-associative operators. AgentMPI accepts them, because

refusing them would exclude most of the operators a harness author wants, and pays for them in depth. Declaring an operator associative when it is not is the agent analogue of lying to `MPI_Op_create`: the program still runs, and the answer silently depends on the agent count.

8.1 Structural and semantic operators

Operators come in two kinds, and the distinction is what makes the depth argument have teeth.

A **structural** operator executes inside the library. It costs no tokens, is exactly reproducible, and its algebra can be established by inspection. AgentMPI predefines eleven: `AMPI_CONCAT` (associative, not commutative), `AMPI_UNION` and `AMPI_BAG` (both), `AMPI_MERGE_JSON` (a deep merge whose scalar conflicts resolve to the lexicographically smaller encoding, which is an arbitrary but total, associative and commutative rule), the numeric and logical operators, `AMPI_MAXLOC` and `AMPI_MINLOC` with MPI's lower-rank tie-break, and `AMPI_VOTE`.

A **semantic** operator cannot execute inside the library, because evaluating it requires a model. AgentMPI handles this with an *upcall*: the library suspends the collective, unwinds to the binding, and hands the calling agent the two operands together with the operator's instructions. The agent evaluates the merge in its own turn, submits the result with `ampi op-submit`, and reissues the identical collective call, which resumes at exactly the step where it stopped. This is the direct analogue of MPI invoking a user function registered through `MPI_Op_create`; the only difference is that the callback runs inside a model turn and is billed in tokens.

Resumption is what makes the upcall practical, and it is why the collective layer is implemented as a persisted step machine rather than a straight-line procedure — the same design LibNBC uses to make collectives nonblocking [24]. Each rank's participation in a collective compiles to a list of send, receive and combine steps, and the program counter lives in the device. One mechanism then buys three properties: collectives that can be nonblocking, collectives that survive an agent process restarting mid-operation, and collectives that can suspend for a model upcall without the other ranks noticing.

`AMPI_VOTE` deserves separate mention because it is the operator with no MPI counterpart at all. Pairwise majority is not associative, so AgentMPI does not implement it pairwise. Instead the reduction accumulates a commutative multiset and a *finalize* step takes the mode, reporting the tally and the agreement fraction rather than only the winner. This monoid-plus-projection shape is the honest way to express quorum operators without misdeclaring their algebra, and it is the agent analogue of algorithm-based fault tolerance [26]: disagreement among redundant executors is surfaced, not averaged away.

Table 2: Allreduce schedules under the agent cost model. R is peak resident tokens; e is operator evaluations on the critical path. The last column is what a semantic operator actually pays, since $\gamma \gg \alpha, \beta$.

schedule	depth d	v	R	e
linear	p	n	$2n$	$p-1$
binomial tree	$2\lg p$	$n \lg p$	$2n$	$\lg p$
recursive doubling	$\lg p$	$2n \lg p$	$2n$	$\lg p$
ring	$2(p-1)$	$2n\frac{p-1}{p}$	$3n/p$	$p-1$
Rabenseifner	$2\lg p$	$2n\frac{p-1}{p}$	$2n/p$	$\lg p$

8.2 A cost model in tokens, turns, and context

The Hockney model that underlies MPI collective analysis charges α per message and β per byte, and the analyses that follow from it are about trading latency terms against bandwidth terms [23, 10]. The substitution for agents is nearly mechanical and one term is new.

Let p be the communicator size and n the payload in tokens. Write α for the per-step dispatch latency, β for the seconds per token transferred, and γ for the seconds an operator evaluation costs when it requires a model turn. Then a schedule of depth d, moving v tokens per rank, with e operator evaluations on the critical path, takes

$$T = d\,\alpha + v\,\beta + e\,\gamma. \tag{1}$$

The fourth quantity, C, is not a cost but a *constraint*: the context window of a single rank. A schedule whose peak residency R — the greatest number of tokens a rank must hold simultaneously — exceeds C is inadmissible at any price.

Table 2 instantiates Equation 1 for the allreduce algorithms AgentMPI implements. The columns are the standard ones with bytes replaced by tokens, plus the two that matter here: e, because when γ is a model turn it dominates every other term by three orders of magnitude, and R, because it decides feasibility rather than speed.

8.3 The context bound decides feasibility

Two rows of Table 2 differ in a way that has no counterpart in the HPC reading of the same table. Recursive doubling and the binomial tree require every rank to materialise the whole vector at every step, so $R = \Theta(n)$ independently of p. The reduce-scatter family — the ring and Rabenseifner's algorithm — reduces one block at a time and can discard a block once it has been forwarded, so $R = \Theta(n/p)$ through the reduction.

In MPI this is a bandwidth argument worth a constant factor, and it is why Rabenseifner's algorithm wins for long messages [37, 39]. Under a context bound it is a feasibility argument:

> For a vector of n tokens over p agents each bounded by a context window of C tokens, an allreduce whose schedule materialises the whole vector is admissible only when $n \lesssim C/2$, while a reduce-scatter-based schedule remains admissible as long as $n \lesssim pC/2$. The reduce-scatter family is therefore not merely faster at scale; past $n \approx C$ it is the only family that can run at all.

This is a claim about schedules, not about a particular implementation, and AgentMPI enforces it rather than merely documenting it: `analyse_allreduce` computes R in closed form and marks a schedule inadmissible when $R > C$, and `select_algorithm` refuses to return an inadmissible schedule. Section 14.3 measures R directly from the step machine and finds the predicted separation.

8.4 The decision function

Open MPI ships a `coll tuned` decision file and MPICH a set of control variables; both select a collective algorithm from message size and communicator size, tuned per machine [39, 17]. AgentMPI's decision function takes two further inputs that HPC does not have — the operator's declared algebra and the rank context bound — and applies three admissibility rules in order:

1. re-association requires a declared associative operator, so a non-associative operator admits only the canonical linear order;
2. block decomposition additionally requires commutativity, because reduce-scatter reorders operands within a block — the same restriction MPICH places on Rabenseifner's algorithm, for the same reason;
3. the peak residency of the schedule must fit the rank context bound.

Among the survivors it minimises Equation 1. It also returns its reasoning: `ampi plan` and `--explain` report every schedule considered, its closed-form cost, and, for each rejected one, why. A harness author who cannot see why their semantic allreduce took p rounds instead of $\lg p$ cannot fix it, and the fix is usually a one-word change to an operator declaration rather than a change to the harness.

9 Context Is a Flow-Control Resource

Long context is not uniformly usable; retrieval quality can depend strongly on position and can degrade in the middle even for long-context models [31]. Therefore AgentMPI does not equate "fits in the advertised window" with "safe to append."

Every destination advertises credits in queued messages, bytes, inline token estimate, and optional provider cost. Admission that would exceed a bound blocks, fails with `RESOURCE_EXHAUSTED`, or externalizes according to a negotiated policy. It never silently truncates.

Large bodies become immutable SHA-256 artifacts. The mailbox contains a bounded summary, media type, size, token estimate, URI/capability, and digest. Resolution is explicit and charged when materialized. This yields a hierarchy: authenticated header, small control value, summary, artifact reference, then selective ranges. Credits return only after dequeue, expiration, or a durable compaction checkpoint.

This mechanism addresses both harness OOM and context "rot." It cannot force a model to attend to loaded text. Adapters should build each prompt from the task's dependency slice rather than replay the full communicator transcript.

10 Lifecycle, Failure, and Synchronization

10.1 Failure detector and fencing

Heartbeats renew an incarnation lease. Expiry creates suspicion under configured timing assumptions; it is not proof. Before a replacement can commit, the old incarnation is fenced. This follows the distinction between failure detection and correctness in asynchronous systems [11].

10.2 Revoke, shrink, agree

On member failure, ordinary operations in the damaged generation return an error. The recovery sequence is:

```
AckFailed(old, failed_set)
Revoke(old)
new, rank_map = Shrink(old)
ok = Agree(new, checkpoint_available)
if ok: redistribute_incomplete_work(new)
```

No strict collective completes over an implicit survivor subset. Shrink creates consistent ordered membership and a fresh generation. Agree provides failure-aware Boolean conjunction; it is not general consensus. FLP still applies to deterministic consensus in a fully asynchronous model with one failure [15].

10.3 Checkpoint and replay

A checkpoint contains generation, agent incarnations, delivered message IDs, unmatched envelopes, artifact digests, collective ordinal, application-state references, and trace position. Hidden model state is not assumed portable. Replay can reproduce harness decisions and substitute captured model outputs; it cannot force a fresh model invocation to generate the same tokens.

10.4 Locks are fenced leases

A time-bounded lease without fencing is unsafe: an executor may pause through expiry, resume, and overwrite

its successor. AgentMPI lock acquisition returns a monotonically increasing token. Protected storage rejects tokens lower than the latest accepted token, adapting lease practice from distributed storage [18]. Long generation should not hold a lock; prefer immutable artifacts, ownership partitioning, and collective merge.

11 Security and Trust Boundaries

MPI commonly assumes a trusted allocation. Internet- or enterprise-facing agents cannot. A remote binding authenticates principals, binds credentials to session/communicator/rank/incarnation, authorizes artifact resolution, prevents cross-run replay, and protects transport integrity/confidentiality.

Received natural language is untrusted input, not authority. A tool-capable executor intersects delegated capability with local policy. Traces retain hashes, schemas, timing, and provenance while redacting secrets and hidden reasoning. Collectives over untrusted model output require validation or robust aggregation at the application layer; crash-stop repair is not Byzantine tolerance.

12 Reference Implementation

The artifact implements a durable semantic reference in Python 3.11+ using only the standard library at runtime. Independent OS processes coordinate through SQLite in WAL mode; no resident broker is needed. This choice favors inspectability and crash persistence over throughput.

12.1 Persistent state and transactions

Eleven tables represent sessions, agents, communicators, messages, posted receives, collective counters/signatures/contributions/results, locks, artifacts, and events. Each transition executes in a short `BEGIN IMMEDIATE` transaction. Message enqueue atomically checks mailbox bytes, assigns the next stream sequence, inserts the envelope, and appends a trace event. Receive publishes a posted-receive row, repeatedly searches the matching index, atomically marks one row acknowledged, charges context, removes the posted receive, and emits the observed match.

The matching query filters communicator, destination, pending state, optional source, and optional tag, then orders eligible rows by admission time, source, and sequence. The index is `(comm, dst, state, src, tag, created)`. Synchronous send polls the durable row until receive marks it acknowledged; ready send verifies a compatible posted receive.

12.2 Collective reference implementation

Each rank obtains a communicator-global ordinal and persists an operation signature and contribution. A conflicting signature marks the instance failed so waiters report mismatch. When every live member has contributed, one transaction computes a rank-ordered deterministic result and inserts it idempotently; all ranks read the same result row. This centralized algorithm is intentionally a transparent reference. Production bindings replace it with the tree/ring algorithms in Table 1 and validate traces against the reference semantics.

12.3 Artifacts, failure, and traces

Bodies above the inline-token threshold are written to a temporary file, SHA-256 hashed, atomically renamed, and referenced in the message. Resolution recomputes the digest before context charge. Heartbeats update leases; failure marks affected communicators revoked. Shrink inserts a new generation with failed/finalized ranks removed. Lock rows store owner, lease expiry, and monotonic fencing token. Every transition appends a session/rank event with a database sequence, time, kind, and JSON metadata, analogous in purpose to the PMPI profiling layer.

12.4 Implemented and specified surface

The prototype implements standard, buffered, synchronous, and ready sends; wildcard receive and probe; barrier, broadcast, scatter, gather, allgather, reduce, allreduce, and agree; context and mailbox bounds; content-addressed artifacts; heartbeat/failure hooks; revoke/shrink; fenced lock acquisition; a CLI; and trace export. The schema specifies source incarnations, deadlines, capability proofs, and typed payload metadata beyond the SQLite subset.

True asynchronous requests, optimized distributed collectives, dynamic spawn, one-sided windows, network authentication, decentralized control-plane recovery, and Byzantine tolerance remain future conformance levels.

13 Evaluation

13.1 Questions and methodology

We separate six questions:

1. Do tests establish matching, isolation, collective, resource, repair, and fencing invariants?
2. What overhead does the transparent SQLite binding impose?
3. Does artifact spill bound prompt-facing context without changing bytes?
4. Can true independent agents execute a collective protocol?
5. Can failure be identified and work reassigned without a stale commit?
6. Does protocol structure affect task quality after controlling model, information, and inference budget?

Questions 1–3 are deterministic protocol experiments. Questions 4–5 are pilot macrobenchmarks. Question 6 requires replicated, randomized comparison and blinded or external evaluation; we provide the design and do not infer it from one run.

All measurements are generated by committed scripts. Results retain raw samples and the metadata applicable to each experiment. The SQLite microbenchmark uses 20 timed iterations (with two ping-pong warmups); the independent-process campaign performs five allreduces in each of seven tested configurations; the fail-stop experiment is one run. The measured host runs CPython 3.12.3 on Linux and reports eight logical CPUs. Values below describe one host, not a network transport.

13.2 Conformance tests

Sixteen SQLite-binding tests exercise ordered point-to-point/wildcard matching, synchronous rendezvous, rank-consistent collectives, artifact spill and context admission, mailbox backpressure, revoke/shrink, fencing-token takeover, and monotonic audit traces. Adversarial additions cover clock regression, cross-tag ordering, posted-receive priority, unjoined/stale incarnations, forged communicators, communicator-local rank remapping, shrink lineage and idempotence, and fencing persistence after clean release. A mismatch test deliberately enters different collectives at one communicator-global ordinal and verifies that both participants receive a persisted diagnostic rather than hang. Static type and lint checks pass. The artifact also contains explicit failure hooks and a required-invariants checklist. A publication artifact should add property-based interleaving tests and a small TLA+/PlusCal model; Section 16 treats this as open work.

13.3 Microbenchmarks

Table 3 reports medians. Ping-pong includes durable enqueue, matching, acknowledgment, polling, and transactions; it is not a network-only latency. The 8 KiB and 64 KiB payloads crossed the configured token threshold in all 20 trials and therefore measured artifact-reference exchange. Their nearly flat ping-pong time demonstrates bounded inline transport work, while artifact materialization cost is measured separately.

Collective latency grows steeply because all ranks poll one SQLite writer and the reference coordinator serializes contributions. At 32 ranks, median barrier and allreduce are 476 and 489 ms, respectively. This negative result is useful: the binding is appropriate for local harness debugging where model calls take seconds, but not for low-latency or cluster-scale collectives. A broker or direct transport should implement logarithmic/ring algorithms.

Failure injection followed by shrink from eight to seven ranks takes a median 1.36 ms (range 0.622–2.454 ms), excluding failure-detection timeout and application-state restoration. The separation matters: reporting only this number as "agent recovery" would be misleading.

Operation	Scale	Median
Ping-pong	16 B	4.23 ms
Ping-pong	1 KiB	9.15 ms
Ping-pong	8 KiB (artifact descriptor)	16.74 ms
Ping-pong	64 KiB (artifact descriptor)	9.42 ms
Barrier	2 ranks	5.58 ms
Barrier	8 ranks	108 ms
Barrier	32 ranks	476 ms
Allreduce	2 ranks	8.09 ms
Allreduce	8 ranks	113 ms
Allreduce	32 ranks	489 ms
Fail+shrink	8 to 7 ranks	1.36 ms

Table 3: Local SQLite reference-runtime medians, 20 trials. Variance is high; raw samples and p95 values are in the artifact.

13.4 Independent-process scaling and failure

Thread-level trials can hide process attachment and independent SQLite connections. A second driver launches one OS process per rank and executes a strict start barrier, five allreduces, point-to-point fan-in at rank 0, and an exit barrier. All configurations from 2 through 100 ranks completed with correct reduction and fan-in. Median per-rank allreduce time rose from 11.5 ms at two ranks to 101 ms at 32, 258 ms at 64, and 944 ms at 100. Total 100-process wall time was 10.45 s. These results establish functional process scale to the requested rank count; they reinforce, rather than weaken, the conclusion that a single SQLite writer is not a high-performance binding.

The fail-stop driver launched 16 independent processes and terminated rank 7 with exit status 99 after all ranks became ready. Rank 0 published failure while survivors were entering a barrier. All 15 survivors returned `COMM_REVOKED`, independently obtained the identical ordered membership $\{0, \ldots, 6, 8, \ldots, 15\}$, and completed an allreduce of survivor ranks with the correct result 113. The full run took 0.335 s and emitted 92 trace events. This includes process startup and a fixed 0.1 s injection delay, but uses an explicit failure hook rather than heartbeat detection.

13.5 Context result

A synthetic JSON body estimated at 16,253 tokens crossed a 64-token inline threshold. The pending mailbox held a 36-token artifact descriptor. Explicit resolution returned 65,011 bytes whose parsed value was byte-equivalent to the source. Thus reference exchange reduced prompt-facing admission charge by 99.78% for this

payload. This verifies storage and accounting behavior, not model comprehension or quality.

13.6 Translation macrobenchmark

The translation workload uses eight selected public-domain passages from Chapter I of Lewis Carroll's *Alice's Adventures in Wonderland* (Project Gutenberg eBook 11). Rank 0 broadcasts a French style/glossary contract and scatters one passage to each of eight translator agents. Translators gather rank-ordered JSON drafts. A second four-agent communicator broadcasts a review schema and scatters adjacent two-passage windows, including boundary neighbors, then gathers omissions, additions, grammar, glossary, and continuity findings.

Every executor is a separately launched Cursor subagent. It joins a durable session and calls the AgentMPI CLI for broadcast, scatter, gather, and finalize; task and result transfer is therefore through the protocol, not copied from a shared chat. Unique per-rank JSON files, database trace, and the root's gathered result are retained. A one-agent full-selection baseline controls for decomposition, while a no-review condition isolates review.

Metrics include collective completion/rank order, glossary exact match, named entity/number/quote preservation, issue categories, cross-boundary term disagreement, revision acceptance, materialized tokens, and wall time. Literary preference needs blinded bilingual judges; an LLM judge from the same family is reported only as sensitivity analysis.

The completed pilot produced all eight drafts, four two-passage reviews, one review-based synthesis, and a separately prompted one-agent baseline. The draft and review communicators emitted 90 protocol events. All eight applicable glossary checks matched exactly. Reviewers reported 16 actionable issues (14 minor and two moderate) and supplied a revision for every passage. Most importantly, a reviewer identified that our selected source accidentally omitted intervening text between passages 3 and 4; the synthesizer preserved the gap rather than hallucinating a bridge. This makes a dataset defect visible; it is not evidence that the final prose is human-preferred.

Our token estimator charges 1,899 aggregate input tokens for one-copy shards plus eight glossary/style deliveries, versus 6,848 for sending all selected text and style to all eight translators—a 72.3% reduction. This comparison excludes provider prompt caching and reviewer/synthesis cost. The observed 759 s protocol span is dominated by intentionally staggered subagent launch under a concurrency cap and is not a speedup measurement.

13.7 Collaborative software macrobenchmark

The second task builds `minidag`, a dependency-free deterministic DAG execution library. Rank 0 sends a versioned contract and file ownership. Eight producers implement graph, strict parser, scheduler, concurrent executor, CLI, tests, documentation, and an executable example. Three staged reviewers wait on explicit producer `DONE` messages. Rank 12 receives every report, runs integration checks, repairs only cross-module defects, and returns a final record. Every file mutation acquires a named AgentMPI lease and records its fencing token.

The live run exposed a lifecycle failure rather than producing an artificially clean trace: two initial launches were rejected by the executor-concurrency limit. The strict broadcast timed out, rank 11 reported its missing rank-10 dependency, and rank 12 returned a structured `blocked` result listing ranks 5, 10, and 11 instead of integrating partial work. Replacement executors for ranks 5 and 10 then completed their queued tasks; rank 11 resumed after receiving the missing `DONE`. A fresh two-rank recovery session assigned integration to a replacement, which independently passed 19 of 19 tests, compilation, and the documented example. The original blocked record, replacement outputs, recovered report, and 13 distinct software-agent IDs are retained.

This event is useful evidence for explicit dependencies and lifecycle visibility, but it is not a ULFM communicator-repair result: the launch rejections occurred before those ranks joined. Crash repair is evaluated separately in the 16-process experiment above. Neither run establishes superiority over one expert developer.

The comparison against a naive prose-only team is descriptive unless prompts contain equal information. A proper ablation fixes model, sampling, role descriptions, token budget, tools, and task assignments, varying only explicit communication/repair semantics.

13.8 One-hundred-rank Aesop dry run

The artifact also contains 100 prepared Aesop-to-Spanish rank outputs and a collector result with an empty missing-rank set (`experiments/results/cursor_scale.json`). That result does not retain per-rank Cursor launch identifiers, so we conservatively classify it as a protocol/collector dry run and exclude it from the true-subagent count and all model-quality claims. The independently reproducible 100-process experiment above is the supported scale result.

14 Protocol Cost and the Context Bound

This section measures the reference runtime described in Section 8 rather than the earlier prototype bindings. It answers four questions:

Q1. What does the protocol itself cost, relative to the thing it coordinates?

Q2. Do the collective schedules behave as their cost models predict?

Q3. Does the reduce-scatter family actually deliver the residency bound that the feasibility argument depends on?

Q4. With real agents evaluating a semantic operator, does the declared algebra buy the predicted reduction in critical-path depth?

Q1–Q3 are measured with *scripted* ranks: one OS process per rank, doing nothing but calling the protocol. This is deliberate. A model turn costs tens of seconds and varies by an order of magnitude, so a measurement taken with model-driven ranks characterises the model, not the protocol. Q4 is measured with Cursor subagents, one per rank, because it is a question about model behaviour and cannot be answered any other way. Every number below is recomputed from the PAMPI event log in the shipped job databases; none is reported by the code under test.

14.1 Protocol overhead

A ping-pong sweep between two ranks on the SQLite device fits Equation 1's point-to-point terms at $\alpha = 2.48\,\text{ms}$ per step and $\beta = 5.35 \times 10^{-7}\,\text{s}$ per token. Half round-trip is 2.82 ms at 5 tokens and 2.92 ms at 563 tokens — flat, because at these sizes the transaction dominates — and rises to 22.1 ms at 36,405 tokens once the payload is spilled to the artifact store (Figure 1).

The absolute numbers matter less than the ratio. A single model turn in the agent experiments below had a median latency of roughly 170 s. At $\alpha = 2.5\,\text{ms}$, one protocol step costs about 1.5×10^{-5} of one turn. A harness can afford a great many protocol operations per unit of agent work, which is the precondition for treating coordination as something to be specified explicitly rather than minimised.

The sweep also exercises the receiver-driven rendezvous switch. At 4,545 tokens the transfer degrades from eager to rendezvous and the receiver is charged 27 tokens — the structural digest — instead of the payload, a 168× reduction; at 36,405 tokens the same 27-token digest is a 1348× reduction. The body remains addressable and is fetched only if the receiver decides to pay for it.

14.2 Collective schedules

Figure 2 shows barrier and allreduce latency against communicator size for every implemented schedule. Two observations, one expected and one not.

The expected one: step counts follow the analysis exactly. At $p = 32$ the dissemination barrier executes 10 steps against the linear barrier's 62, and the binomial allreduce 15 against the ring's 281.

The unexpected one: fewer steps does not always mean less time on this device. The dissemination barrier is *slower* than the linear barrier at every $p \geq 4$ — 269 ms against 178 ms at $p = 32$ — despite using a sixth of the steps. The reason is that α is not uniform. A dissemination round is a strict alternation of send and blocking receive, so each of its 10 steps pays a full polling round-trip, while the linear barrier's root issues 31 receives that are already satisfied by the time it asks. The Hockney model assumes a single α for every step; on a store-based transport the effective α depends on whether a step waits. We report this because it is exactly the kind of thing a decision function has to be tuned for, and exactly the reason MPI implementations ship measured decision files rather than deriving the choice from step counts.

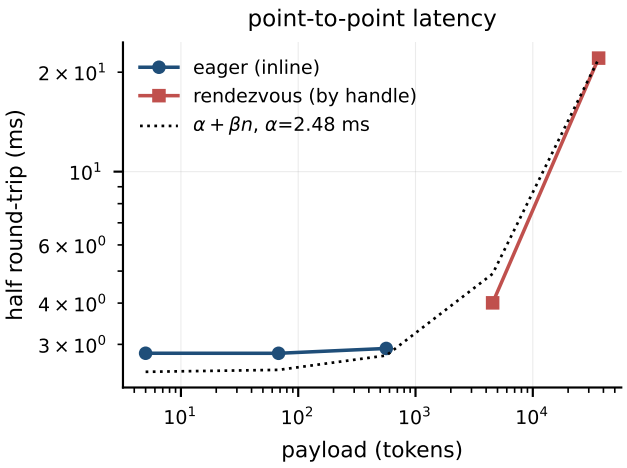

Figure 1: Point-to-point half round-trip against payload size. Latency is flat while messages are delivered inline and rises once the receiver-driven switch sends them by reference instead; the dotted line is the fitted $\alpha + \beta n$ model.

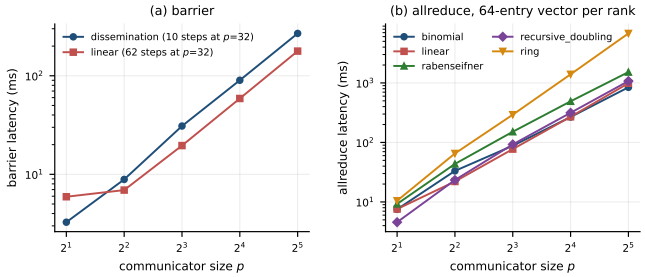

Figure 2: Collective latency against communicator size, scripted ranks, SQLite device. Step counts follow the analysis, but the dissemination barrier's fewer, strictly-serialised steps lose to the linear barrier's pipelined ones.

14.3 Peak context residency

Figure 3 reports the quantity the feasibility argument turns on: the largest number of tokens a rank must hold simultaneously, measured directly from the collective step machine rather than inferred. Each rank contributes 256 keyed entries, so the reduced vector grows

with p — a weak scaling sweep, which is the regime a harness is actually in when it adds agents to cover more material.

The separation is unambiguous and matches Table 2. Recursive doubling and the binomial tree hold $1.50n$ tokens at every p tested, from $p = 4$ to $p = 32$. Recursive-halving reduce-scatter falls from $0.68n$ at $p = 4$ to $0.09n$ at $p = 32$. In absolute terms the reduce-scatter residency is essentially *constant* — 10.8 k, 10.7 k, 11.0 k and 11.7 k tokens at $p = 4, 8, 16, 32$ — because n/p is held fixed by the weak-scaling design, while the doubling family's residency grows with the vector to 197 k tokens at $p = 32$.

That last figure is the point. At $p = 32$ the reduced vector is 127,776 tokens. Under a 128 k-token context window, an allreduce that materialises the vector at every rank cannot run: it needs 197 k tokens resident, more than the window, and no amount of patience fixes it. Reduce-scatter completes the same reduction with 9% of the vector resident. The decision function reaches this conclusion in closed form before any message is sent, and reports it: `ampi plan` marks recursive doubling inadmissible with the reason "peak residency of 197033 tokens exceeds the rank context limit".

14.4 One-sided operations under contention

Uncontended window operations are cheap and flat: put 0.09 ms, get 0.04 ms, atomic fetch-and-add 0.08 ms, lease acquisition 0.09–0.12 ms, across $p = 2$ to $p = 32$. What grows is contention. With every rank performing a read-modify-write on one shared cell, failed compare-and-swaps rise from 0 at $p = 2$ to 2, 4, 19 and 100 at $p = 4, 8, 16, 32$ (Figure 4) — superlinear, as the Universal Scalability Law's coherency term predicts [20].

For a byte-oriented store those retries are nearly free. For an agent they are not: a lost compare-and-swap on an artifact means the agent must re-read, re-apply its edit, and in the semantic case re-think it, at model cost. This is the quantitative case for the window API offering `win-claim` and leases rather than only put and get: a work item claimed by compare-and-swap is claimed once, whereas a work item taken by convention is taken by however many agents happen to look at the same moment.

14.5 Two transports behind one interface

An abstract device interface with one implementation behind it is an indirection, not an interface. The reference runtime therefore ships two devices: the SQLite device, in which every rank opens one durable file and a single writer serialises everything, and a filesystem device with no coordinator on the data path, in which a message is a file written to a temporary name and `rename(2)`-d into the destination rank's mailbox, and a claim is a second rename that only one racing claimant can win. They bracket the design space real MPI implementations occupy: one behaves like a shared or offloaded fabric where a single agent observes all traffic, the other like a distributed-memory fabric where nobody does.

Both pass the same device conformance suite, which tests the four concurrency properties the layers above depend on rather than functionality: that `match` hands a record to exactly one claimant under eight concurrent claimants, that `cas` rejects a stale version and admits exactly one of six racing writers, that leases exclude and expire, and that the ticket counter never repeats across 150 concurrent increments.

Writing that suite found a real defect in the SQLite device. Its `match` originally issued the select and the claiming update as two autocommit statements; under contention two ranks could both read the same posted message before either wrote, and 40 work items were handed out 181 times. This is precisely the duplicated-work failure that multi-agent harnesses report, in the one component whose entire job is to prevent it, and it was invisible until the suite ran eight claimants against it concurrently. We mention it because the lesson generalises: the coordination guarantees an agent protocol advertises are exactly the ones that will be quietly wrong unless something tests them under real concurrency.

14.6 Full experimental design

For a publishable quality claim, we propose a preregistered factorial study:

- topology: independent vote, star, tree, sparse DAG, all-to-all;
- protocol: implicit shared transcript versus AgentMPI typed messages;
- context: full replication versus summaries/artifact-on-demand;
- failure: none, crash before result, crash after result/before ACK, straggler, duplicated retry;
- scale: 1, 2, 4, 8, 16 agents;
- task: separable translation, dependency-rich software, factual reasoning, and adversarial correlated-error cases.

Use multiple model families and at least three random seeds per cell; randomize task-instance assignment; report paired effects with bootstrap confidence intervals and mixed-effects models over task/model. Cost-normalize comparisons by input/output tokens and tool calls. Report all failures and family-wise correction for multiple primary outcomes. Protocol overhead is measured with deterministic replay of captured outputs; semantic benefit is measured with identical transport/control conditions.

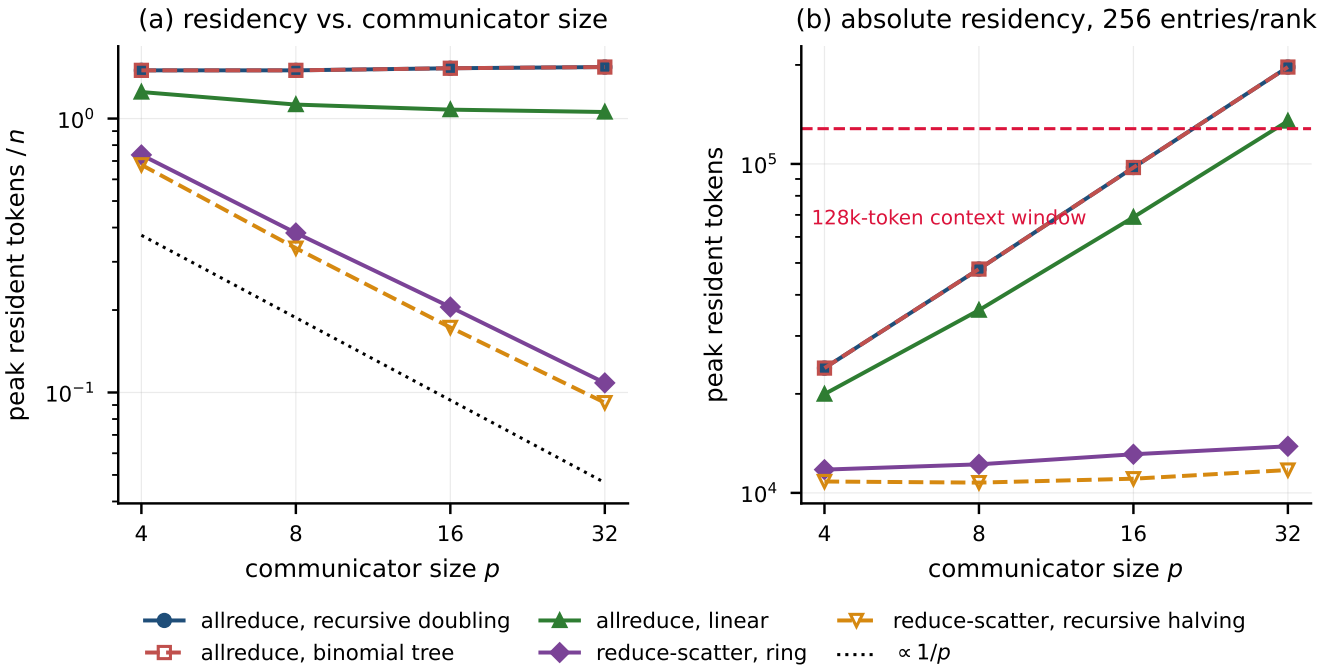

Figure 3: Peak resident tokens per rank during an allreduce of a keyed vector, measured from the step machine, weak scaling at 256 entries per rank. Left: residency relative to the vector size n. Right: absolute residency against a 128 k-token context window; above the dashed line the operation cannot be performed at all. The reduce-scatter family is the only one whose reduction phase residency falls with p.

15 Discussion

15.1 What becomes portable

AgentMPI can make a harness's communication graph, matching assumptions, phase boundaries, resource policy, failure behavior, and trace schema portable. It gives runtime implementers optimization points and conformance tests. It also creates a vocabulary for comparing systems: "synchronous send" has a match completion condition, while "semantic accept" is separate.

15.2 What remains application-specific

Prompts, role policies, decomposition quality, validation, side-effect transactions, and domain semantics remain above the interface. This is a feature. MPI does not choose the numerical method; AgentMPI should not choose the deliberation method.

15.3 Actor and workflow comparison

Actors provide isolated state, asynchronous mailboxes, supervision, and location transparency [22, 2, 8]. Workflow engines provide durable task state and retries. AgentMPI can be implemented over either, but adds communicator-scoped matching and standardized collective intent. Conversely, it should borrow supervision and durable idempotency rather than reproduce decades of runtime engineering.

15.4 Standardization path

A real standard requires at least two independent interoperable implementations, a conformance suite, wire/version negotiation, a stable governance forum, and application experience. The MPI Forum's multi-party process is as important a precedent as its API. Version 0.1 is a proposal to test, not a declaration that a new standard already exists.

16 Limitations and Threats to Validity

Single implementation. SQLite validates semantics locally but says little about broker, HTTP, or cross-organization interoperability. The control plane and filesystem are single-host trust/failure domains.

Central collective reference. Measured collective scaling reflects write serialization and polling, not the tree/ring algorithms proposed for production. It is useful as a lower- engineering-complexity reference, not a performance baseline for MPI.

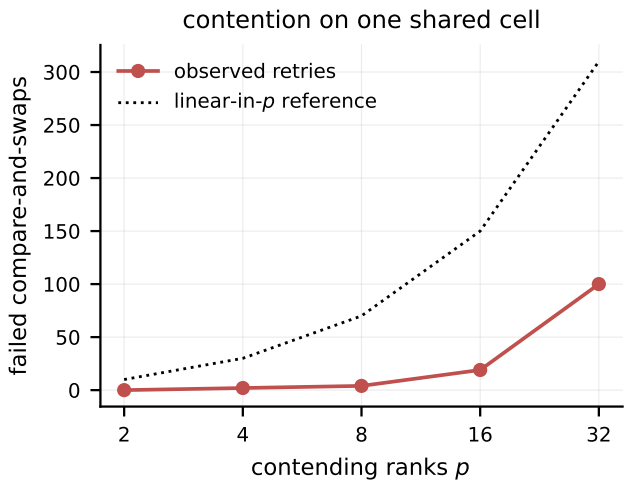

Figure 4: Failed compare-and-swaps on a single shared window cell as the number of contending ranks grows. Retries are cheap for bytes and expensive for agents, which is why the protocol offers atomic claim rather than leaving mutual exclusion to convention.

Failure model. Hooks model crash-stop ranks. Heartbeat timeout can falsely suspect a paused executor. Provider-wide outages, malicious agents, inconsistent clocks, and coordinator failure need separate treatment.

Context estimate. The tokenizer-independent byte heuristic is not exact for any provider. Externalization controls admission but not attention, retrieval, or "lost in the middle" effects.

Pilot agent evidence. Cursor subagents can share a model family, training biases, environment, and scheduler. Agent outputs are not independent samples. Small runs cannot support universal quality claims, and agent-generated review is correlated.

Task representativeness. Translation is highly separable; software integration has a particular DAG. Other domains may need shared world models, low-latency interaction, or stronger transactional state.

Specification/implementation gap. The wire schema and specification include capabilities, incarnations, and future request semantics beyond the persisted subset. Conformance levels and capability negotiation must make this gap machine-visible.

Formal verification. Tests cover concrete schedules but not all interleavings. The artifact now includes a TLA+ safety model for matching, communicator-global collective slots, crash/revoke/repair, and persistent fencing. With three agents and two bounded operations, TLC generated 45,073,807 states (11,152,584 distinct) to depth 24 without an invariant violation [28]. An earlier version produced a counterexample when crash mutated membership in place; the repaired model keeps generation membership immutable and records failure separately. This remains a bounded design check, not an implementation-refinement proof or an unbounded liveness result. Credits, cancellation, and concrete transport refinement remain open; stateful model-based testing is also planned in the QuickCheck tradition [13].

17 Conclusion

Multi-agent systems need fewer framework metaphors and more explicit contracts. MPI's durable lesson is not that every agent should impersonate an HPC process; it is that portable applications become possible when communication contexts, matching, completion, collectives, progress, and implementation freedom are specified precisely.

AgentMPI applies that lesson while exposing the differences that matter: nondeterministic semantic work, expensive invocation, bounded and imperfect context, elastic executors, ordinary failure, external side effects, and cross-domain trust. The protocol and SQLite runtime show that every central concept can be tied to an executable transition and trace. The measurements also show the limits of that implementation. The next milestone is not more API surface: it is independent bindings, model-checked invariants, and cost-controlled multi-provider experiments.

A Normative Core API Sketch

```
Session.Init(required_capabilities) -> Session
Session.GetPsets() -> [ProcessSet]
Comm.Create(group, attributes) -> Comm
Comm.Split(color, key) -> Comm

Send(value, dest, tag, comm, mode) -> Status
Recv(source|ANY, tag|ANY, comm) -> (value, Status)
Isend/Irecv(...) -> Request
Test/Wait/WaitAny/WaitSome/WaitAll(requests)
Probe/MatchedProbe(source, tag, comm)

Barrier(comm)
Bcast(value, root, comm)
Scatter(values, root, comm)
Gather(value, root, comm)
Allgather(value, comm)
Reduce(value, operator, root, comm)
Allreduce(value, operator, comm)
Alltoall(values, comm)
Scan(value, operator, comm)
NeighborAllgather(value, graph_comm)

AckFailed(comm, failures)
Revoke(comm)
Shrink(comm) -> (new_comm, rank_map)
Agree(comm, flag) -> bool

Artifact.Put(bytes, metadata) -> Ref
Artifact.Get(ref, ranges?) -> bytes
```

```
Context.Compact(checkpoint)
Lock.Acquire(name, ttl) -> (lease, fence)
```

B Required Invariants

1. No envelope is delivered outside its session, communicator, generation, and destination.
2. One at-most-once admitted message is exposed at most once.
3. Fixed-stream messages do not overtake.
4. Only a compatible posted receive consumes a data envelope.
5. A strict collective completes only after every current member enters the identical signature.
6. A failed-member collective never silently succeeds on a subset.
7. Membership changes advance generation and return an explicit map.
8. Mailbox/context accounting never exceeds negotiated limits.
9. Artifact resolution verifies content identity.
10. An expired owner cannot commit after a newer fencing token.
11. Every completion has a causally linked trace event.
12. Protocol control messages cannot be consumed by wildcard application receives.

C Artifact Layout

`SPEC.md` Normative draft and conformance levels.
`schemas/` Canonical JSON transport envelope.
`src/agentmpi/` Runtime, model, and CLI.
`tests/` Conformance and failure tests.
`scripts/run_benchmarks.py` Raw microbenchmark generator.
`experiments/tasks/` Public-domain inputs and exact executor protocols.
`experiments/results/` Raw samples, traces, per-rank products, and gathered outputs.
`paper/` This source and bibliography.

References

[1] A2A Project. *Agent2Agent Protocol Specification, Version 1.0*, 2026.

[2] Joe Armstrong. Erlang. *Communications of the ACM*, 53(9):68–75, 2010.

[3] Jonas Becker, Lars Benedikt Kaesberg, Andreas Stephan, Jan Philip Wahle, Terry Ruas, and Bela Gipp. Stay focused: Problem drift in multi-agent debate. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 5068–5102. Association for Computational Linguistics, 2026.

[4] Wesley Bland, George Bosilca, Aurélien Bouteiller, Thomas Hérault, and Jack J. Dongarra. Post-failure recovery of MPI communication capability: Design and rationale. *International Journal of High Performance Computing Applications*, 27(3):244–254, 2013.

[5] Wesley Bland, Aurélien Bouteiller, Thomas Hérault, Joshua Hursey, George Bosilca, and Jack J. Dongarra. An evaluation of user-level failure mitigation support in MPI. *Computing*, 95(12):1171–1184, 2013.

[6] George Bosilca, Aurélien Bouteiller, Franck Cappello, Samir Djilali, Gilles Fédak, Cécile Germain, Thomas Hérault, Pierre Lemarinier, Oleg Lodygensky, Frédéric Magniette, Vincent Néri, and Anton Selikhov. MPICH-V: Toward a scalable fault tolerant MPI for volatile nodes. In *Proceedings of SC '02*, 2002.

[7] Jehoshua Bruck, Ching-Tien Ho, Shlomo Kipnis, Eli Upfal, and Derrick Weathersby. Efficient algorithms for all-to-all communications in multiport message-passing systems. *IEEE Transactions on Parallel and Distributed Systems*, 8(11):1143–1156, 1997.

[8] Sergey Bykov, Alan Geller, Gabriel Kliot, James R. Larus, Ravi Pandya, and Jorgen Thelin. Orleans: Cloud computing for everyone. In *Proceedings of the 2nd ACM Symposium on Cloud Computing*, 2011.

[9] Ralph H. Castain, David Solt, Joshua Hursey, and Aurélien Bouteiller. PMIx: Process management for exascale environments. *Parallel Computing*, 79:9–29, 2018.

[10] Ernie Chan, Marcel Heimlich, Avi Purkayastha, and Robert van de Geijn. Collective communication: theory, practice, and experience. *Concurrency and Computation: Practice and Experience*, 19(13):1749–1783, 2007.

[11] Tushar Deepak Chandra and Sam Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267, 1996.

[12] Hyeong Kyu Choi, Xiaojin Zhu, and Sharon Li. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems*, 2025.

[13] Koen Claessen and John Hughes. Quickcheck: A lightweight tool for random testing of haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming*, pages 268–279, 2000.

[14] Andrew Estornell and Yang Liu. Multi-LLM debate: Framework, principles, and interventions. In *Advances in Neural Information Processing Systems*, volume 37, pages 28938–28964, 2024.

[15] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

[16] Foundation for Intelligent Physical Agents. *FIPA ACL Message Structure Specification*, 2002.

[17] Edgar Gabriel, Graham E. Fagg, George Bosilca, Thara Angskun, Jack J. Dongarra, Jeffrey M. Squyres, Vishal Sahay, Prabhanjan Kambadur, Brian Barrett, Andrew Lumsdaine, Ralph H. Castain, David J. Daniel, Richard L. Graham, and Timothy S. Woodall. Open MPI: Goals, concept, and design of a next generation MPI implementation. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, volume 3241 of *Lecture Notes in Computer Science*, pages 97–104, 2004.

[18] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proceedings of the 12th ACM Symposium on Operating Systems Principles*, pages 202–210, 1989.

[19] William Gropp, Ewing Lusk, Nathan Doss, and Anthony Skjellum. A high-performance, portable implementation of the MPI message passing interface standard. *Parallel Computing*, 22(6):789–828, 1996.

[20] Neil J. Gunther. *Guerrilla Capacity Planning*. Springer, 2007. Universal Scalability Law: contention and coherency terms.

[21] Debra Hensgen, Raphael Finkel, and Udi Manber. Two algorithms for barrier synchronization. *International Journal of Parallel Programming*, 17(1):1–17, 1988.

[22] Carl Hewitt, Peter Bishop, and Richard Steiger. A universal modular actor formalism for artificial intelligence. In *Proceedings of the 3rd International Joint Conference on Artificial Intelligence*, pages 235–245, 1973.

[23] Roger W. Hockney. *The Science of Computer Benchmarking*. SIAM, 1996. Origin of the widely used α–β communication cost model.

[24] Torsten Hoefler, Andrew Lumsdaine, and Wolfgang Rehm. Implementation and performance analysis of non-blocking collective operations for MPI. In *Proc. ACM/IEEE Conference on Supercomputing (SC'07)*, 2007.

[25] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.

[26] Kuang-Hua Huang and Jacob A. Abraham. Algorithm-based fault tolerance for matrix operations. *IEEE Transactions on Computers*, C-33(6):518–528, 1984.

[27] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.

[28] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[29] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large scale language model society. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[30] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. More agents is all you need. *Transactions on Machine Learning Research*, 2024.

[31] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[32] Message Passing Interface Forum. MPI: A message passing interface. In *Proceedings of Supercomputing '93*, pages 878–883, 1993.

[33] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*, November 2023.

[34] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 5.0*, June 2025.

[35] Model Context Protocol Contributors. *Model Context Protocol Specification*, 2025.

[36] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. Chatdev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[37] Rolf Rabenseifner. Optimization of collective reduction operations. In *Computational Science – ICCS 2004*, volume 3036 of *Lecture Notes in Computer Science*, pages 1–9, 2004.

[38] Andries Petrus Smit, Nathan Grinsztajn, Paul Duckworth, Thomas D. Barrett, and Arnu Pretorius. Should we be going MAD? a look at multi-agent debate strategies for LLMs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 45883–45905, 2024.

[39] Rajeev Thakur, Rolf Rabenseifner, and William Gropp. Optimization of collective communication operations in MPICH. *International Journal of High Performance Computing Applications*, 19(1):49–66, 2005.

[40] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*, 2024.

[41] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the First Conference on Language Modeling*, 2024.